

Birzeit University
Faculty of Engineering & Technology
Department of Electrical & Computer Engineering

Text-to-Video Retrieval System Using Deep Learning

Computer Vision Project 2

prepared by:
Sondos Qasarwa 1210259
Sarah Hassouneh 1210068

Instructor: Mr. Aziz Qaroush

April 24, 2026

Contents

1 Introduction **2**

2 Related Work **3**
2.1 Vision-Language Pre-training . . . 3
2.1.1 CLIP Architecture . . . 3
2.2 Video Representation Learning . . . 3
2.2.1 Frame-based Approaches . . . 3
2.3 Temporal Aggregation Strategies . . . 4
2.3.1 Simple Pooling Methods . . . 4
2.3.2 Learned Temporal Attention . . . 4
2.3.3 Transformer-based Temporal Pooling . . . 5
2.4 Text-to-Video Retrieval Systems . . . 5
2.4.1 CLIP4Clip (2021) . . . 5
2.4.2 X-CLIP (2022) . . . 5

3 Proposed Method **7**
3.1 System Overview . . . 7
3.2 CLIP-based Feature Encoding . . . 7
3.3 Frame Sampling Strategy . . . 8
3.4 Frame-level Adapter Module . . . 8
3.5 Video Representation via Mean Pooling . . . 8
3.6 Training Objective . . . 9

4 Dataset and Evaluation **10**

5 Experimental Results **12**
5.1 Implementation Details . . . 12
5.1.1 Sampling Strategy . . . 12
5.1.2 Aggregation method . . . 12
5.1.3 Lightweight Learnable Heads. . . . 12
5.2 Zero-Shot Ablation Study . . . 13
5.2.1 Effect of Multi-frame Representation and Normalization . . . 13
5.2.2 Number of Sampled Frames . . . 13
5.2.3 Frame Sampling Strategy . . . 14
5.2.4 Temporal Aggregation Methods . . . 14
5.3 Learned Heads . . . 15
5.4 Final Results on MSR-VTT 1k-A Test Set . . . 15
5.5 Comparison with Related Work . . . 15

6 Discussion & Conclusion **17**
6.1 Key Findings . . . 17
6.2 Limitations . . . 17
6.3 Conclusion . . . 18

1 Introduction

Text-to-video retrieval is a process that aims to retrieve specific videos from a database based on natural language queries. Success in this task means returning videos that are semantically relevant to the query text. However, this introduces a fundamental challenge: the semantic gap between text and video modalities.

While text can be understood as a sequence of words and video as a sequence of frames, this framing oversimplifies the problem. Unlike standard sequence-to-sequence tasks in NLP, where both input and output share the same modality, text-to-video retrieval requires bridging textual and visual inputs. Furthermore, video frames cannot be treated as independent entities. The most important information often lies in the motion and actions depicted across sequences of frames, meaning that spatial relationships alone are insufficient, thus temporal modelling becomes essential. This multimodal and temporal nature makes the problem considerably more complex.

In this project, we build a text-to-video retrieval system based on contrastive learning and vision–language models, specifically CLIP. We use CLIP as a fixed backbone and extend it to the video domain through an integrated, staged pipeline. Leveraging CLIP's strong image–text representations, we construct video-level embeddings from sampled frame features and introduce a lightweight trainable adapter that applies small refinements to frame-level representations while preserving the pre-trained semantics. This design enables effective retrieval without relying on computationally heavy modelling.

2 Related Work

2.1 Vision-Language Pre-training

In this work we aim to explore contrastive visual language p retraining for text-to-video retrieval. Contrastive learning is a training approach that allows a model to learn similarities between inputs, where some input pairs are treated as positive samples and others as negative samples. The model therefore pulls positive pairs closer together while pushing unmatched pairs apart. In recent years, contrastive learning has been extensively adopted for model pre-training, which involves learning representations from large amounts of unlabelled data. This approach has been further transferred to multimodal representations, particularly in supervised vision–language pre-training settings, where a shared embedding space is learned for multimodal image–text pairs [1].

This essentially enabled models to learn semantically meaningful visual representations that generalize beyond task-specific classifiers. By aligning visual representations with linguistic descriptions in a shared space, contrastive models are able to learn transferable representations that are less sensitive to dataset-specific biases and visual variability. An additional advantage of contrastive learning is its flexibility across different levels of training strategies. Contrastive models can be employed in a zero-shot manner without relying on specific training data.

2.1.1 CLIP Architecture

CLIP stands for Contrastive Language Image Pre-training, the method was initially presented by OpenAI. CLIP jointly trains an image encoder and a text encoder to predict the correct pairings within a batch of (image, text) training examples. This approach emphasizes that learning from natural language pairs has several potential advantages over classical training methods [2]. CLIP applies symmetric contrastive loss with prompt engineering and natural language supervision to represent the image-text pairs.

2.2 Video Representation Learning

Video representation learning involves extracting compact feature embeddings from video data that preserve semantically meaningful information for downstream tasks such as classification, retrieval, and captioning. Various approaches exist for video encoding, ranging from 3D convolutional networks that process spatio-temporal volumes directly, to frame-based methods that leverage pre-trained image encoders by treating videos as sequences of independent frames.

2.2.1 Frame-based Approaches

Videos consist of dense temporal sequences containing hundreds or thousands of frames, making exhaustive processing computationally prohibitive. Frame sampling strategies address this challenge by selecting a representative subset of frames that preserve essential visual information while reducing computational overhead. Since pre-trained models like CLIP operate on static images, frame selection becomes crucial for video understanding. Common sampling strategies include uniform sampling, which selects frames at regular intervals; random sampling, which introduces stochasticity; and motion-based sampling, which prioritizes frames with high inter-frame variation to capture dynamic content.

2.3 Temporal Aggregation Strategies

Although CLIP achieves impressive results on various image-text tasks including classification and retrieval, video understanding introduces an additional challenge that CLIP was not designed to handle: temporal feature modeling. CLIP's architecture operates on individual static frames, treating each image independently without considering temporal relationships across the video sequence. Consequently, multiple approaches have been developed to incorporate temporal reasoning into CLIP's framework. Given a video with T frames, CLIP extracts frame-level embeddings $\mathbf{v}_t \in \mathbb{R}^d$ for $t = 1, \ldots, T$, and the goal is to aggregate these into a unified video representation $\mathbf{v}_{\text{video}} \in \mathbb{R}^d$.

2.3.1 Simple Pooling Methods

The most straightforward approaches to aggregating frame-level features extracted by CLIP involve simple pooling operations. These methods collapse the temporal dimension by applying a reduction function across all frame embeddings.

Mean Pooling computes the element-wise average of all frame features, producing a single video-level representation:

$$\mathbf{v}_{\text{video}} = \frac{1}{T} \sum_{t=1}^{T} \mathbf{v}_t \tag{1}$$

where $\mathbf{v}_t \in \mathbb{R}^d$ represents the CLIP embedding of frame t, T is the total number of frames, and $\mathbf{v}_{\text{video}}$ is the resulting video representation.

Max Pooling selects the maximum activation value across all frames for each feature dimension:

$$\mathbf{v}_{\text{video}}[i] = \max_{t=1}^{T} \mathbf{v}_t[i], \quad \forall i \in \{1, \ldots, d\} \tag{2}$$

where $\mathbf{v}_{\text{video}}[i]$ denotes the i-th dimension of the video embedding.

Simple pooling methods primary strength lies in their simplicity and computational efficiency, as they require no additional parameters or training procedures. These methods naturally handle videos of varying lengths by producing fixed-dimensional representations regardless of the number of input frames, providing robustness across diverse video datasets. However, the most critical limitation is the complete loss of temporal information, as pooling operations discard the sequential ordering of frames, making it impossible to capture motion patterns, action progression, or temporal dependencies that are often crucial for video understanding.

2.3.2 Learned Temporal Attention

To overcome limitations of simple pooling methods, several works have introduced learned temporal attention mechanisms that assign different importance weights to frames based on their relevance. The simplest approach uses linear attention, where frame weights are computed as:

$$\alpha_t = \frac{\exp(\mathbf{w}^T \mathbf{v}_t)}{\sum_{j=1}^{T} \exp(\mathbf{w}^T \mathbf{v}_j)} \tag{3}$$

Additionally, more expressive variants employ multi-layer perceptrons (MLPs) for non-linear scoring functions. These methods preserve temporal information by weighting frames based on learned importance, offering a balance between the simplicity and complexity.

2.3.3 Transformer-based Temporal Pooling

Transformer encoders can be applied to frame sequences for temporal modeling through self-attention mechanisms. Self-attention allows frames to attend to each other, capturing temporal dependencies:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V} \tag{4}$$

where $\mathbf{Q} = \mathbf{V}\mathbf{W}_Q$, $\mathbf{K} = \mathbf{V}\mathbf{W}_K$, and $\mathbf{V} = \mathbf{V}\mathbf{W}_V$ are learned projections of the frame features $\mathbf{V} = [\mathbf{v}_1, \ldots, \mathbf{v}_T]$.

Multi-head self-attention can capture different temporal patterns simultaneously. Following transformer encoding, the final video representation can be obtained through CLS token pooling or mean pooling across the transformed frame features.

2.4 Text-to-Video Retrieval Systems

Recent years multiple works have adopted contrastive learning, specifically the CLIP models for videos, by incorporating multiple aggregation strategies.

2.4.1 CLIP4Clip (2021)

CLIP4Clip was among the first models to adapt CLIP for video retrieval tasks. Their pipeline consists of three primary components: a text encoder, a video encoder that outputs frame-level representations, and a similarity calculator. The authors explored three distinct similarity measures to compare text and video representations. The first is a *parameter-free* approach that uses mean pooling over frame embeddings. The second is a *sequential* approach that employs either a Transformer or LSTM encoder to model temporal dependencies across frames. The third is a *tight* similarity type that diverges from the previous two by introducing cross-modal interaction through a Transformer Encoder, similar to the approach in [3]. The authors concluded that the parameter-free and sequential strategies achieved better results by preventing overfitting and balancing computational cost with fewer parameters.

CLIP4Clip demonstrated state-of-the-art performance across various benchmark datasets and exhibited significant zero-shot transfer capabilities. We will compare our system against CLIP4Clip as a baseline in our experimental evaluation.

2.4.2 X-CLIP (2022)

X-CLIP was later introduced with the idea of employing a multi-grained contrastive learning framework to improve video-text retrieval performance by learning correspondences at multiple levels of granularity. The model considers fine-grained (frame-word), coarse-grained (video-sentence), and cross-grained (video-word, sentence-frame) contrasts, where the relevance of individual words and frames to the query is dynamically weighted based on their importance [4].

X-CLIP architecture consists of three primary components: a text encoder, a frame encoder, and a temporal encoder. Input sentences are processed by the text encoder to generate both coarse-grained (sentence-level) and fine-grained (word-level) textual representations. On the visual side, input videos are sampled into frames and fed into the frame encoder to produce frame-level representations. These frame-level representations

are then processed by a temporal encoder that employs cross-frame attention mechanisms to capture temporal relationships across the video sequence. X-CLIP then computes similarity scores across different granularities, through an Attention Over Similarity Matrix (AOSM) module. We will compare our system against X-CLIP as an additional baseline in our experimental evaluation.

3 Proposed Method

3.1 System Overview

Our system builds upon CLIP, a large-scale image–text model, and extends it to the text-to-video retrieval task through a lightweight and efficient adaptation strategy. Since CLIP is originally designed to align individual images with text, we adapt it to handle videos by constructing video-level representations from frame-level features while preserving the pre-trained vision–language alignment.

Given an input video, we first uniformly sample a fixed number of frames to reduce computational overhead while maintaining sufficient visual coverage. Each sampled frame is independently encoded using the frozen CLIP image encoder. To adapt these image-level representations to the video-text retrieval task, we apply a lightweight adapter module to the frame features, enabling task-specific refinement with minimal additional parameters. The adapted frame embeddings are then aggregated using mean pooling to form a single video-level representation. In parallel, the text query is encoded using the frozen CLIP text encoder. During training, only the adapter parameters are updated usin the contrastive objective, while all CLIP encoder parameters remain frozen.

At retrieval time, video embeddings are extracted using the frozen CLIP encoders and the learned adapter. Given a text query, the corresponding text embedding is computed using the CLIP text encoder, and videos are ranked based on cosine similarity between video and text embeddings.

Figure 3.2 illustrates the overall architecture of the proposed CLIP-based framework for text-to-video retrieval.

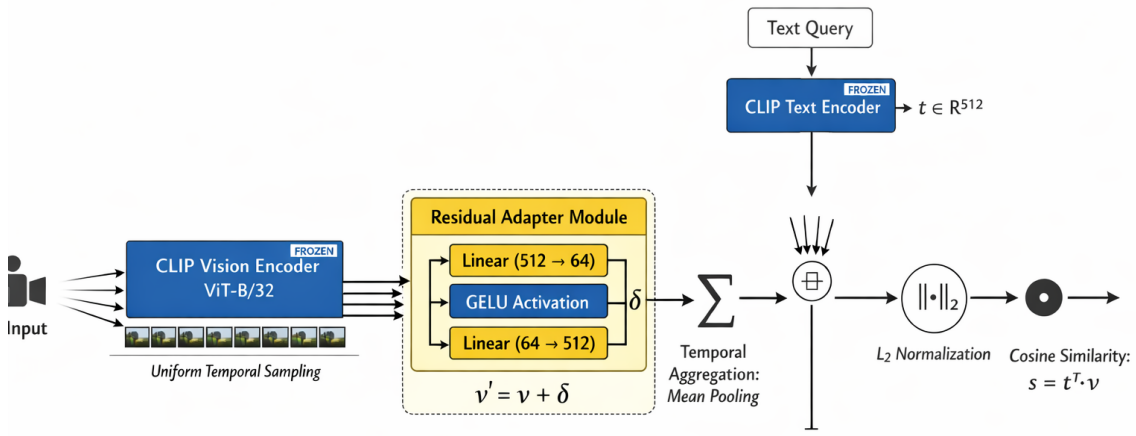


Figure 1: The overall architecture of the proposed CLIP-based framework for text-to-video retrieval.

3.2 CLIP-based Feature Encoding

Let t denote a text query and v a video. We uniformly sample K frames from v, denoted as $\{f_k\}_{k=1}^{K}$.

Text Encoder. The text query is encoded using the frozen CLIP text encoder:

$$\mathbf{e}_t = E_{\text{text}}(t) \in \mathbb{R}^d, \tag{5}$$

where $d = 512$ in our implementation.

Frame Encoder. Each sampled frame is independently encoded using the frozen CLIP vision encoder:

$$\mathbf{e}_{f_k} = E_{\text{img}}(f_k), \quad \mathbf{e}_{f_k} \in \mathbb{R}^d. \tag{6}$$

Using CLIP enables us to leverage semantically rich visual representations without introducing additional visual backbones or temporal encoders.

3.3 Frame Sampling Strategy

Processing all frames of a video is computationally expensive and often redundant. We therefore uniformly sample a fixed number of frames per video. This strategy significantly reduces computational cost while maintaining sufficient visual coverage, as CLIP frame-level embeddings already capture high-level semantic information.

3.4 Frame-level Adapter Module

Since CLIP is originally trained on image–text pairs, we introduce a lightweight adapter module to better align frame-level visual features with the video–text retrieval task, while keeping the CLIP encoders frozen.

Given a frame embedding $\mathbf{e}_{f_k}$, the adapter is applied independently to each frame:

$$\tilde{\mathbf{e}}_{f_k} = \mathbf{e}_{f_k} + \mathcal{F}(\mathbf{e}_{f_k}), \tag{7}$$

where $\mathcal{F}(\cdot)$ denotes a two-layer MLP with a bottleneck structure:

$$\mathcal{F}(\mathbf{x}) = \mathbf{W}_2 \, \text{GELU}(\mathbf{W}_1 \mathbf{x}). \tag{8}$$

The residual formulation preserves CLIP's pre-trained representations while enabling task-specific adaptation with a small number of trainable parameters. Importantly, the adapter operates at the frame level and does not introduce explicit temporal modeling.

3.5 Video Representation via Mean Pooling

To obtain a single video-level representation, we aggregate the adapted frame embeddings using mean pooling:

$$\mathbf{e}_v = \frac{1}{K} \sum_{k=1}^{K} \tilde{\mathbf{e}}_{f_k}. \tag{9}$$

The resulting video embedding is ℓ_2-normalized prior to similarity computation. Mean pooling provides a simple yet effective mechanism for extending CLIP from image-level to video-level representations without introducing additional temporal parameters.

3.6 Training Objective

We compute the similarity between a video embedding $\mathbf{e}_v$ and a text embedding $\mathbf{e}_t$ using cosine similarity:

$$s(v,t) = \frac{\mathbf{e}_v^\top \mathbf{e}_t}{\|\mathbf{e}_v\| \, \|\mathbf{e}_t\|}. \tag{10}$$

The model is optimized using a standard symmetric InfoNCE contrastive loss that encourages higher similarity for matched video–text pairs than for mismatched pairs within a batch. Given a batch of N video–text pairs, the text-to-video loss is defined as:

$$\mathcal{L}_{t2v} = -\frac{1}{N} \sum_{i=1}^{N} \log \frac{\exp(s(v_i, t_i)/\tau)}{\sum_{j=1}^{N} \exp(s(v_j, t_i)/\tau)}, \tag{11}$$

where τ denotes the temperature parameter. The final training objective is computed symmetrically as

$$\mathcal{L} = \tfrac{1}{2}(\mathcal{L}_{t2v} + \mathcal{L}_{v2t}), \tag{12}$$

with other samples in the batch serving as in-batch negatives.

During training, only the adapter parameters are updated, while all CLIP encoder parameters remain frozen, enabling parameter-efficient adaptation.

4 Dataset and Evaluation

MSR-VTT Dataset

To perform our experimental studies, we mainly use the MSR-VTT dataset. It is composed of 10,000 videos, each with a length that ranges from 10 to 32 seconds and 200,000 captions—with each video having 20 different captions. We depend on this dataset since it is widely used in this specific task and provides a stable benchmark for us to test our work.The dataset features diverse video content spanning multiple categories, including entertainment, sports, cooking, news, and educational content, providing a comprehensive benchmark for evaluating retrieval performance across varied domains.

Following common practice in text-to-video retrieval, we evaluate our method on the widely adopted 1k-A test split, which consists of 1,000 videos with 20 captions per video. Although MSR-VTT provides an official test split, the 1k-A split is used by most prior work and enables fair comparison with existing methods.

The remaining videos are used for training and validation, with approximately 6,000 videos for training and 497 videos for validation. All hyperparameter tuning and model selection are performed exclusively on the validation set, and the 1k-A test split is used only for final evaluation. During training and validation, we use one caption per video to construct video–text pairs.

Evaluation Metrics

To assess retrieval quality, we will use the following measures:

1. **Recall@K (R@K)**: Measures the proportion of relevant videos that appear within the top-K retrieved results for a given query. We evaluate recall at K=1, 5, and 10 to capture performance at different retrieval depths.

   Formula:
   $$R@K = \frac{\text{Number of relevant videos in top-K}}{\text{Total relevant videos in database}} \tag{13}$$

2. **Median Rank (MedR)**: Represents the median position at which the first relevant video appears across all queries. This metric is robust to outliers with lower values indicate better performance.

   Formula:
   $$\text{MedR} = \text{median}(\text{rank}_1, \text{rank}_2, \ldots, \text{rank}_Q) \tag{14}$$

   where rank_q is the position of the first relevant video for query q, and Q is the total number of queries.

3. **Mean Rank (MeanR)**: Computes the average position at which the first relevant video appears across all queries. Lower values indicate better performance.

   Formula:
   $$\text{MeanR} = \frac{1}{Q}\sum_{q=1}^{Q} \text{rank}_q \tag{15}$$

   where rank_q is the position of the first relevant video for query q, and Q is the total number of queries.

4. **Mean Average Precision (mAP)**: The main metric for video retrieval. It averages AP across all queries, giving overall system performance. It computes:

   - Precision at each relevant retrieval
   - Average Precision (AP) for each query
   - mAP = mean(AP over all queries)

   Formula:

$$AP = \frac{1}{R} \times \sum_{k} (\text{Precision@}k \times \text{rel}(k)) \tag{16}$$

$$mAP = \frac{1}{Q} \times \sum_{q} AP(q) \tag{17}$$

   where Q = total number of queries and R = total relevant videos, rel(k) = 1 if item at rank k is relevant

5. **Retrieval Time**: Measures the computational efficiency of the retrieval system, which is especially important when working with large-scale video databases. This metric is defined as the average time required to process a single query during online search. It includes the time needed to compute similarity scores between the query representation and all video representations in the database, as well as any subsequent ranking or re-ranking operations performed at query time.

   Note that offline preprocessing steps, such as video feature extraction, frame sampling, descriptor aggregation, dimensionality reduction, and index construction, are excluded from this measurement.

5 Experimental Results

5.1 Implementation Details

The experiments are conducted in two stages. First, we study zero-shot design choices for extending CLIP from images to videos by encoding sampled video frames independently with CLIP and aggregating their representations into a single video-level embedding. We analyze the effect of frame sampling, normalization, and temporal aggregation strategies. Second, we evaluate lightweight trainable heads built on top of the best zero-shot configuration. All ablations are conducted on the validation set, and the final model is evaluated once on the MSR-VTT 1k-A test set.

We use the CLIP ViT-B/32 model as the vision and text backbone in all experiments. The CLIP image encoder is applied independently to sampled video frames, and the CLIP text encoder is used to encode text queries. Both encoders produce 512-dimensional embeddings. All CLIP parameters are initialized from the publicly available pre-trained weights and remain frozen throughout training.

5.1.1 Sampling Strategy

Several frame sampling strategies are evaluated in the ablation study. Uniform sampling selects frames evenly spaced across the video duration. Random sampling selects frames uniformly at random. Gaussian sampling biases frame selection toward the temporal center of the video by sampling from a normal distribution. Motion-based sampling selects frames exhibiting larger inter-frame visual changes, computed using frame-wise intensity differences on downsampled grayscale frames.

5.1.2 Aggregation method

Given frame-level CLIP features $\{f_i\}_{i=1}^{N}$, temporal aggregation produces a single video embedding. Mean pooling computes the average feature across frames, while max pooling takes the element-wise maximum. In addition, a parameter-free similarity-based self-attention pooling strategy is evaluated: normalized frame features are used to compute a frame–frame similarity matrix, row-wise softmax yields attention weights, and attended frame features are averaged to produce the final video representation.

5.1.3 Lightweight Learnable Heads.

To improve video–text retrieval without fine-tuning CLIP, lightweight learnable heads are trained on top of frozen CLIP frame features. For each video, N frames are sampled and encoded independently using the CLIP image encoder, producing frame-level embeddings $\{f_i\}_{i=1}^{N}$ with $f_i \in \mathbb{R}^D$. A head then maps these frame embeddings to a single video-level embedding $v \in \mathbb{R}^D$, which is L2-normalized and trained using a CLIP-style symmetric contrastive loss. Only the head parameters are updated during training, while the CLIP image and text encoders remain frozen.

Two heads are evaluated. (1) **Learned frame-weighting pooling** assigns a scalar score to each frame using a single linear layer and converts the scores to weights using a softmax across frames. The video embedding is obtained as a weighted sum of the

original frame embeddings followed by normalization. (2) **Adapter + mean pooling** applies a small bottleneck MLP independently to each frame embedding with a residual connection ($f_i' = f_i + \mathrm{MLP}(f_i)$). The adapted frame embeddings are then aggregated using fixed mean pooling to form the video embedding. In both cases, the head outputs a single D-dimensional representation per video.

We evaluated a small set of adapter hyperparameters on the validation set. The reported results use the selected configuration with a bottleneck dimension of 64 and a learning rate of 1×10^{-4}.

5.2 Zero-Shot Ablation Study

5.2.1 Effect of Multi-frame Representation and Normalization

To quantify the limitations of directly applying image-based CLIP representations to video–text retrieval, we first evaluate a single-frame baseline in which only the center frame of each video is encoded. We then extend this baseline by aggregating features from multiple sampled frames using mean pooling. As reported in Table 1, incorporating multiple frames leads to a substantial improvement in retrieval performance, with R@1 increasing from 26.76% to 37.83%. Furthermore, applying per-frame ℓ_2 normalization before aggregation yields an additional performance gain, achieving an R@1 of 38.63%. These results indicate that multi-frame representations are essential for effective video–text retrieval and that normalization improves the robustness of frame-level feature aggregation.

Table 1: Effect of multi-frame representation and per-frame normalization on the MSR-VTT validation set.

Configuration	R@1	R@5	R@10	MedR	MeanR	Time (ms)
1 frame (center)	26.76	52.52	62.98	5	40.3	0.04
8 frames, mean pool (no norm)	37.83	62.98	71.83	3	25.2	0.02
8 frames, mean pool + per-frame ℓ_2 norm	**38.63**	**62.98**	**72.03**	**3**	**25.2**	0.02

5.2.2 Number of Sampled Frames

In this experiment, we vary the number of sampled frames while keeping the sampling strategy fixed to uniform sampling and using mean pooling for temporal aggregation.

Table 2: Effect of the number of sampled frames on retrieval performance (validation set).

Frames (N)	R@1	R@5	R@10	MedR	Time (ms)
4	36.22	63.98	70.82	3	0.02
8	**38.63**	62.98	72.03	3	0.02
12	37.22	**64.59**	**74.45**	3	0.02
16	38.23	63.78	73.44	3	0.02

As shown in Table 2, increasing the number of frames from 4 to 8 results in a clear improvement in R@1, indicating that incorporating additional temporal context benefits

video–text retrieval. Performance saturates beyond 8 frames, with only marginal gains or small fluctuations observed when increasing the number of frames to 12 or 16. While higher frame counts slightly improve R@10, the improvements in R@1 are limited. These results suggest that a moderate number of frames is sufficient to capture the relevant visual content for retrieval, and we therefore use 8 frames in subsequent experiments.

5.2.3 Frame Sampling Strategy

This experiment evaluates the effect of different frame sampling strategies, while fixing the number of sampled frames to 8 and using mean pooling for temporal aggregation.

Table 3: Comparison of different frame sampling strategies (validation set).

Sampling Strategy	R@1	R@5	R@10	MedR	Time (ms)
Uniform	**38.63**	62.98	72.03	3	0.02
Random	33.80	61.57	69.22	3	0.02
Gaussian	34.81	**64.79**	71.23	3	0.02
Motion-based	34.00	61.77	71.23	3	0.02

As shown in Table 3, uniform sampling achieves the best overall performance, obtaining the highest R@1 of 38.63%. Random sampling leads to a noticeable drop in performance, indicating that randomly selected frames may miss informative visual content. Gaussian sampling slightly improves R@5 compared to uniform sampling but does not outperform it in terms of R@1. Motion-based sampling performs comparably to random sampling and does not yield additional benefits in this setting. Overall, these results suggest that uniformly sampling frames across the video provides a robust and effective strategy for zero-shot video–text retrieval.

5.2.4 Temporal Aggregation Methods

This experiment compares temporal aggregation methods in the zero-shot setting while fixing the sampling strategy to uniform sampling and the number of frames to N=8.

Table 4: Comparison of temporal aggregation methods in the zero-shot setting (validation set).

Aggregation	R@1	R@5	R@10	MedR	MeanR	Time (ms)
Mean pooling	**38.63**	62.98	72.03	3	25.2	0.02
Max pooling	24.14	51.71	61.57	5	38.7	0.02
Similarity-based self-attention (non-learned)	37.83	62.98	72.03	3	25.3	0.02

Table 4 reports the effect of different temporal aggregation methods. Mean pooling provides the strongest overall performance in this zero-shot setting, achieving the best R@1 of 38.63%. Max pooling performs substantially worse, suggesting that selecting only the strongest activations across frames is not sufficient for capturing video-level semantics. Similarity-based self-attention pooling performs close to mean pooling but does not provide consistent improvements and incurs a small additional overhead. These results indicate that simple averaging of frame-level CLIP features is a strong baseline for zero-shot video–text retrieval.

5.3 Learned Heads

After identifying the best zero-shot configuration, lightweight learnable heads are evaluated to further improve video–text retrieval performance without fine-tuning CLIP. All learned heads operate on frozen CLIP frame features and are trained using the same contrastive objective.. Table 5 compares these learned heads against the best zero-shot baseline on the validation set.

Table 5: Comparison of learned temporal aggregation methods on the validation set.

Method	Training	R@1	R@5	R@10	MedR	MeanR	Time (ms)
Best zero-shot (mean pooling)	–	38.63	62.98	72.03	3	25.2	0.02
Learned frame-weighting pooling	Yes	39.03	63.58	73.24	3	24.4	0.02
Adapter (mean pooling)	Yes	**42.45**	**69.21**	**78.27**	**2**	**15.46**	0.03

As shown in Table 5, both learned heads improve over the zero-shot baseline, confirming that limited supervision can enhance temporal modeling without modifying the CLIP encoders. Learned frame-weighting pooling yields a modest improvement, suggesting that learning frame importance provides some benefit over fixed aggregation. In contrast, the adapter-based head achieves a substantially larger gain, improving R@1 from 38.63% to 42.45% and reducing the median rank from 3 to 2. This indicates that adapting frame-level representations before aggregation is more effective than learning aggregation weights alone. Importantly, both approaches introduce negligible inference overhead.

5.4 Final Results on MSR-VTT 1k-A Test Set

The final model configuration is selected based on validation performance. Specifically, the adapter-based head with mean pooling, uniform frame sampling, and $N = 8$ frames achieved the best retrieval accuracy on the validation set. This configuration is therefore fixed and evaluated once on the MSR-VTT 1k-A test set to avoid test set overfitting.

Table 6: Final retrieval performance on the MSR-VTT 1k-A test set.

Method	R@1	R@5	R@10	MedR	Time (ms)
Adapter (final model)	**33.40**	59.80	69.70	3	0.03

5.5 Comparison with Related Work

As shown in Table 7, recent approaches such as CLIP4Clip and X-CLIP report the strongest results on the MSR-VTT benchmark. Their performance gains mainly stem from explicitly enhancing CLIP with additional temporal modeling and fine-grained cross-modal alignment mechanisms, including sequential transformers and multi-grained contrastive objectives. In contrast, our method maintains a simpler design and avoids introducing additional architectural complexity. Although it does not achieve the highest recall scores, it demonstrates that competitive text-to-video retrieval performance can be obtained without relying on heavy temporal modeling or multi-level contrastive alignment.

Table 7: Comparison with state-of-the-art methods on MSR-VTT test set.

Method	**Year**	**R@1**	**R@5**	**R@10**	**MedR**	**MeanR**
CLIP4Clip (mean)	2021	42.1	71.9	81.4	2	15.7
CLIP4Clip (sequential Transformer)	2021	44.5	71.4	81.6	2	15.3
X-CLIP (ViT-B/32)	2022	46.1	73.0	83.1	2	13.2
X-CLIP (ViT-B/16)	2022	49.3	75.8	84.8	2	12.2
VideoCLIP	2021	30.9	55.4	66.8	–	–
Our Method		33.40	59.80	69.70	3	26

6 Discussion & Conclusion

6.1 Key Findings

The experimental evaluation on the MSR-VTT validation set leads to several key findings. First, we demonstrate that pre-trained image-text models such as CLIP can be directly applied to text-to-video retrieval without task-specific training. This highlights the power of contrastive pre-training, as CLIP achieves competitive zero-shot performance with minimal setup, making it particularly valuable in scenarios with limited computational resources or training time.

Our ablation studies reveal that multi-frame representation, normalization, frame sampling strategy, and temporal aggregation methods all significantly affect zero-shot retrieval performance. Multi-frame representations are essential for capturing sufficient video information, with 8 frames proving adequate for the MSR-VTT dataset. Surprisingly, despite evaluating multiple sampling strategies, uniform sampling consistently outperformed more complex alternatives such as random, Gaussian, and motion-based sampling. Similarly, simple mean pooling achieved the best results among temporal aggregation methods, suggesting that more sophisticated zero-shot aggregation strategies may be prone to overfitting on this dataset.

Second, we show that introducing lightweight learnable heads on top of frozen CLIP encoders further improves retrieval accuracy. Both learned frame-weighting pooling and adapter-based approaches outperform the zero-shot baseline. The adapter-based head yields the largest gains, improving R@1 from 38.63% to 42.45% on the validation set and reducing the median rank from 3 to 2. This demonstrates that adapting frame-level features before aggregation is more effective than learning aggregation weights alone, while maintaining computational efficiency and preserving CLIP's pre-trained representations.

6.2 Limitations

Despite achieving competitive text-to-video retrieval performance, our system obtains a recall@1 of only 33.4% on the MSR-VTT test set, which is substantially lower than state-of-the-art methods such as X-CLIP (49.3%) and CLIP4Clip (44.5%). While this gap can be partially attributed to our focus on efficiency, simplicity, and minimal computational overhead, it highlights the trade-off between model complexity and retrieval accuracy.

The experimental setup was conducted under limited computational resources, restricting our ability to explore more sophisticated architectures or larger adapter configurations. Although our ablation studies demonstrate that learnable components significantly improve performance over zero-shot CLIP, resource constraints prevented extensive hyperparameter tuning or evaluation of deeper adapter networks that might further close the performance gap. Additionally, our evaluation is limited to a single dataset (MSR-VTT), and performance on other video-text retrieval benchmarks remains unexplored.

Nevertheless, a central objective of this work was to develop a parameter-efficient and computationally lightweight approach to video-text retrieval. Our method prioritizes fast inference, low memory footprint, and minimal training requirements, making it suitable for resource-constrained scenarios where state-of-the-art methods may be impractical.

6.3 Conclusion

In this work, we explored the use of contrastive learning through a pre-trained CLIP model to efficiently address text-to-video retrieval. We demonstrated that frame sampling strategies and temporal aggregation methods significantly improve zero-shot retrieval accuracy, enabling effective adaptation of an image-based model to video understanding. We then introduced lightweight learnable components—specifically, adapter modules and learned pooling heads—to capture temporal information while keeping CLIP encoders frozen. This approach balances model complexity, computational efficiency, and retrieval performance, achieving a final R@1 of 33.40% on the MSR-VTT test set with minimal trainable parameters.

References

[1] F. Sammani, B. Joukovsky, and N. Deligiannis, "Visualizing and understanding contrastive learning," 2023. [Online]. Available: https://arxiv.org/abs/2206.09753v3

[2] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark *et al.*, "Learning transferable visual models from natural language supervision," in *International Conference on Machine Learning (ICML)*. PMLR, 2021, pp. 8748–8763.

[3] H. Luo, L. Ji, M. Zhong, Y. Chen, W. Lei, N. Duan, and T. Li, "Clip4clip: An empirical study of clip for end to end video clip retrieval," *Neurocomputing*, vol. 508, pp. 293–304, 2022.

[4] Y. Ma, G. Xu, X. Sun, M. Yan, J. Zhang, and R. Ji, "X-clip: End-to-end multi-grained contrastive learning for video-text retrieval," in *Proceedings of the 30th ACM International Conference on Multimedia (MM)*, 2022, pp. 638–647.